DATA ENGINEERING

Ujian Tengah Semester — Proyek RAG

Retrieval-Augmented Generation (RAG)

Membangun Sistem Tanya-Jawab Cerdas
Berbasis Dokumen Lokal

Mata Kuliah	Data Engineering
Program Studi	D4 Teknologi Rekayasa Perangkat Lounak
Bobot Nilai	30% dari Nilai Akhir
Durasi Proyek	2 Minggu
Sifat	Kelompok (2–3 Mahasiswa)
Metode Pengumpulan	GitHub Repository + Demo Video

⚠ Baca seluruh panduan sebelum memulai pengerjaan ⚠

1. Deskripsi Proyek

Pada proyek UTS ini, mahasiswa diminta untuk membangun sebuah sistem Retrieval-Augmented Generation (RAG) yang mampu menjawab pertanyaan pengguna berdasarkan dokumen yang diunggah. Sistem ini merupakan salah satu aplikasi nyata dari Data Engineering modern yang menggabungkan pipeline data, pemrosesan bahasa alami, dan basis data vektor.

Proyek ini dirancang agar mahasiswa memahami dan mengimplementasikan alur kerja data end-to-end: mulai dari pengambilan data mentah, transformasi, penyimpanan dalam vector database, hingga retrieval dan generation jawaban menggunakan Large Language Model (LLM).

1.1 Apa itu RAG?

RAG (Retrieval-Augmented Generation) adalah teknik yang menggabungkan dua kemampuan utama:

- Retrieval — mencari dokumen atau potongan teks yang relevan dari basis data
- Generation — menggunakan LLM untuk menghasilkan jawaban berdasarkan konteks yang ditemukan

> 💡 *RAG memungkinkan LLM menjawab pertanyaan spesifik dari dokumen Anda sendiri, bukan hanya dari data pelatihan model. Ini sangat berguna untuk chatbot perusahaan, sistem FAQ, dan asisten dokumen internal.*

1.2 Relevansi dalam Data Engineering

Sistem RAG melibatkan komponen-komponen inti Data Engineering:

Komponen RAG	Konsep Data Engineering yang Diterapkan
Document Loader	Data Ingestion — mengambil data dari berbagai sumber (PDF, TXT, Web)
Text Splitter	Data Transformation — memecah dokumen menjadi chunk yang optimal
Embedding Model	Feature Engineering — mengubah teks ke representasi vektor numerik
Vector Database	Data Storage — menyimpan dan mengindeks embedding secara efisien
Retriever	Data Query — mencari dokumen relevan berdasarkan similarity search
LLM + Prompt	Data Presentation — menghasilkan output yang mudah dipahami pengguna

2. Tujuan Pembelajaran

Setelah menyelesaikan proyek ini, mahasiswa diharapkan mampu:

- 🔧 Membangun pipeline data RAG end-to-end menggunakan Python
- 📄 Melakukan document loading, chunking, dan embedding secara programatik
- 🗄️ Menggunakan vector database (ChromaDB, FAISS, atau lainnya) untuk menyimpan dan mencari embedding
- 🤖 Mengintegrasikan LLM API (OpenAI, Gemini, Groq, Ollama, atau lainnya) ke dalam pipeline
- 📊 Mengevaluasi performa sistem RAG secara kuantitatif dan kualitatif
- 📝 Mendokumentasikan proyek secara profesional di GitHub

3. Persyaratan Proyek

3.1 Spesifikasi Teknis Wajib

⚠️ *Mahasiswa BEBAS memilih tools dan library RAG (LangChain, LlamaIndex, atau dari scratch), namun semua komponen di bawah ini WAJIB ada.*

No	Komponen	Spesifikasi Minimum	Bobot
1	Document Loader	Mampu memuat minimal 2 jenis format (PDF + TXT/CSV/DOCX/Web)	**10%**
2	Text Splitter	Chunking dengan chunk_size dan chunk_overlap yang dapat dikonfigurasi	**10%**
3	Embedding	Gunakan embedding model (bebas: OpenAI, HuggingFace, Ollama, dsb.)	**15%**
4	Vector Store	Gunakan vector database (ChromaDB, FAISS, Pinecone, Qdrant, dsb.)	**15%**
5	Retriever	Similarity search dengan top-k minimal 3 dokumen relevan	**10%**
6	LLM Integration	Integrasikan LLM untuk menghasilkan jawaban (API atau lokal/offline)	**15%**
7	User Interface	Antarmuka sederhana: CLI, Streamlit, Gradio, atau web app	**10%**

8	Evaluasi	Uji dengan minimal 10 pertanyaan, catat relevansi jawaban	**10%**
9	Dokumentasi	README lengkap + diagram arsitektur + cara menjalankan	**5%**

3.2 Topik Dokumen (Pilih Salah Satu)

Kelompok dapat memilih salah satu domain berikut sebagai basis dokumen sistem RAG:

Domain	Contoh Dokumen
📚 Akademik & Pendidikan	Buku teks, modul kuliah, jurnal, silabus
🏥 Kesehatan & Medis	Leaflet obat, panduan kesehatan, FAQ klinik
⚖️ Hukum & Regulasi	Peraturan kampus, UU/perda lokal, SOP organisasi
🛒 Bisnis & E-Commerce	Katalog produk, FAQ pelanggan, kebijakan toko online
🌾 Pertanian & Lingkungan	Panduan budidaya, data cuaca, regulasi lingkungan
💻 Teknologi & IT	Dokumentasi teknis, manual software, tutorial

✅ *Topik lain di luar daftar ini diperbolehkan dengan persetujuan dosen sebelum hari ke-3 proyek dimulai.*

4. Rekomendasi Tech Stack

Berikut adalah rekomendasi tools yang dapat digunakan. Mahasiswa BEBAS menggunakan tools lain yang setara, namun wajib mendokumentasikan alasan pemilihannya di README.

4.1 Framework RAG

Framework	Deskripsi & Instalasi
LangChain	Framework lengkap, dokumentasi kaya, cocok untuk pemula — pip install langchain langchain-community
LlamaIndex	Fokus pada document indexing & retrieval canggih — pip install llama-index
From Scratch	Gunakan langsung: sentence-transformers + FAISS/ChromaDB + requests ke LLM API

4.2 Pilihan LLM (Prioritaskan yang Gratis)

> 💰 *Tidak ada kewajiban menggunakan LLM berbayar. Ollama (offline) adalah pilihan terbaik jika tidak memiliki akses internet stabil.*

Provider	URL / Cara Akses	Status	Catatan
Groq API	console.groq.com	**Gratis**	Rate limit, support Llama 3 & Mixtral, sangat cepat
Google Gemini	aistudio.google.com	**Gratis***	Gemini 1.5 Flash gratis dengan kuota harian
Ollama	ollama.ai	**100% Gratis**	Jalankan Llama 3 / Mistral secara offline di laptop
HuggingFace	huggingface.co/inference	**Gratis***	Model open-source tertentu tersedia gratis
OpenAI API	platform.openai.com	**Berbayar**	Kredit trial tersedia untuk akun baru

4.3 Vector Database

Vector DB	Tipe	Keunggulan
ChromaDB	Lokal / In-memory	Mudah digunakan, cocok untuk pemula, tidak perlu server
FAISS	Lokal / In-memory	Dikembangkan Meta, sangat cepat untuk dataset besar
Qdrant	Lokal / Cloud	Fitur lengkap, support filter metadata yang kuat
Pinecone	Cloud (SaaS)	Managed service, tier gratis tersedia, mudah di-scale

5. Arsitektur Sistem yang Diharapkan

Sistem RAG yang dibangun harus mengimplementasikan dua pipeline utama:

5.1 Pipeline Indexing (Offline — Jalankan Sekali)

Pipeline ini memproses dan menyimpan dokumen ke dalam vector database.

Dokumen Sumber (PDF / TXT / Web / CSV)

↓

Langkah 1: Document Loader → Baca & parse dokumen

↓

Langkah 2: Text Splitter → Bagi menjadi chunk-chunk kecil (chunking)

↓

Langkah 3: Embedding Model → Ubah teks ke vektor numerik

↓

Vector Database → Simpan embedding + metadata dokumen

5.2 Pipeline Query (Online — Setiap Pertanyaan Pengguna)

Pipeline ini dijalankan setiap kali pengguna mengajukan pertanyaan baru.

Pertanyaan Pengguna (User Query)

↓

Langkah 1: Query Embedding → Ubah pertanyaan ke vektor

↓

Langkah 2: Similarity Search → Cari top-k chunk paling relevan dari Vector DB

↓

Langkah 3: Prompt Construction → Gabungkan konteks + pertanyaan dalam prompt

↓

Langkah 4: LLM Generation → LLM menghasilkan jawaban berdasarkan konteks

↓

Jawaban Terstruktur kepada Pengguna

6. Timeline & Milestone (14 Hari)

Hari	Fase	Target Capaian
Hari 1–3	Perencanaan	Pilih topik domain, tentukan stack, buat GitHub repo, tulis deskripsi proyek (1 paragraf)
Hari 4–7	Indexing Pipeline	Document loader + text splitter + embedding + penyimpanan ke vector DB berjalan tanpa error
Hari 8–11	Query Pipeline	Retriever + LLM integration + antarmuka pengguna minimal sudah berfungsi end-to-end

Hari 12–13	📊 **Evaluasi & Tuning**	Uji 10+ pertanyaan, optimasi chunk size/overlap, tulis laporan evaluasi, perbaiki bug
Hari 14	📦 **Finalisasi**	README lengkap, diagram arsitektur, demo video 3–7 menit, push ke GitHub, kumpulkan link

7. Ketentuan Pengumpulan

7.1 Deliverables yang Harus Dikumpulkan

A. GitHub Repository (Wajib)

- Kode sumber lengkap, terstruktur, dan dapat dijalankan ulang
- File requirements.txt atau environment.yml dengan versi library yang jelas
- README.md: deskripsi proyek, arsitektur, cara instalasi, cara menjalankan, contoh tanya-jawab
- Diagram arsitektur sistem (gambar PNG/JPG di dalam folder docs/)
- File .env.example — JANGAN pernah push API key asli ke GitHub!

B. Laporan Evaluasi — PDF, Maks. 5 Halaman (Wajib)

- Deskripsi sistem dan stack yang dipilih beserta alasannya
- Tabel evaluasi 10 pertanyaan: pertanyaan, jawaban sistem, jawaban ideal, skor relevansi 1–5
- Analisis kelemahan sistem dan saran perbaikan ke depan

C. Demo Video — 3–7 Menit (Wajib)

- Format MP4 atau link YouTube / Google Drive (pastikan dapat diakses)
- Tampilkan: cara menjalankan sistem, minimal 5 sesi tanya-jawab, penjelasan singkat arsitektur

7.2 Struktur Folder GitHub yang Direkomendasikan

```
project-rag-uts/
├── data/                # Dokumen sumber (PDF, TXT, dll.)
├── src/
│   ├── indexing.py      # Pipeline indexing
│   ├── query.py         # Pipeline query & retrieval
│   ├── embeddings.py    # Konfigurasi embedding model
│   └── utils.py         # Fungsi helper
├── ui/                  # Antarmuka pengguna (Streamlit / CLI)
├── docs/
│   └── arsitektur.png   # Diagram arsitektur sistem
├── evaluation/
```

```
│   └── hasil_evaluasi.xlsx
├── .env.example        # Template env variable (tanpa nilai asli)
├── requirements.txt
└── README.md
```

📁 *Hindari nama file seperti 'coba.py', 'final2.py', atau 'fix_beneran.py'. Gunakan nama yang deskriptif dan profesional.*

8. Rubrik Penilaian

Aspek Penilaian	Kriteria Detail	Bobot
Fungsionalitas Sistem	Pipeline indexing & query berjalan lengkap; semua 9 komponen wajib ada dan berfungsi	**30%**
Kualitas Kode	Kode bersih, terstruktur, ada komentar, bisa dijalankan ulang oleh orang lain tanpa modifikasi besar	**20%**
Evaluasi & Analisis	Laporan evaluasi 10+ pertanyaan lengkap dan analisis kritis terhadap hasil sistem	**20%**
Demo & Presentasi	Demo lancar, mampu menjelaskan arsitektur dengan jelas, menjawab pertanyaan dosen	**15%**
Dokumentasi README	README lengkap, ada diagram arsitektur, cara instalasi dan menjalankan mudah dipahami	**15%**

8.1 Bonus Poin (Maks. +10 Poin)

⭐ *Fitur berikut tidak wajib, tetapi dapat menambah nilai. Bonus diberikan atas dasar implementasi yang berjalan, bukan sekadar ada di kode.*

Fitur Bonus	Deskripsi
Hybrid Search	Kombinasikan semantic search (vector) + keyword search (BM25) untuk retrieval lebih akurat
Reranking	Implementasikan cross-encoder reranker untuk mengurutkan ulang hasil retrieval
Chat History	Sistem mampu mengingat percakapan sebelumnya (multi-turn conversation)
Source Citation	Setiap jawaban menyertakan referensi sumber (nama file + nomor halaman/chunk)

Deployment	Deploy ke Streamlit Cloud / HuggingFace Spaces / Railway dan cantumkan URL yang aktif

9. Integritas Akademik & Ketentuan Lainnya

🚫 *Pelanggaran integritas akademik dapat mengakibatkan nilai 0 dan sanksi akademik sesuai peraturan institusi.*

9.1 Yang Diperbolehkan

- Menggunakan kode dari tutorial atau dokumentasi resmi, dengan mencantumkan sumber di komentar
- Berdiskusi antara kelompok tentang konsep umum (tidak berbagi kode secara langsung)
- Menggunakan AI asisten (ChatGPT, Claude, Copilot) sebagai alat bantu belajar, BUKAN untuk menulis seluruh kode proyek
- Menggunakan library dan framework open-source apapun

9.2 Yang TIDAK Diperbolehkan

- Menyalin kode kelompok lain secara keseluruhan
- Menyuruh atau membayar orang lain untuk mengerjakan proyek
- Menggunakan proyek RAG dari internet tanpa modifikasi dan kontribusi yang signifikan
- Push API key asli ke GitHub repository (gunakan .env dan .gitignore)

9.3 Konsekuensi Keterlambatan Pengumpulan

Keterlambatan	Pengurangan Nilai
1–24 jam setelah deadline	**-10 poin**
25–48 jam setelah deadline	**-20 poin**
Lebih dari 48 jam	**Nilai 0 — tidak diterima**

10. FAQ — Pertanyaan yang Sering Diajukan

? Apakah harus menggunakan LangChain?

💬 Tidak. Bebas menggunakan LlamaIndex, membangun dari scratch, atau framework lainnya. Jelaskan pilihan Anda di README.

? Bolehkah menggunakan LLM lokal tanpa API key?

Ya, sangat disarankan! Gunakan Ollama untuk menjalankan model Llama 3 atau Mistral secara offline dan 100% gratis.

? Berapa minimal dokumen yang harus dimuat?

Minimal 2 format dokumen berbeda, dengan total konten yang cukup menghasilkan 10 pasang pertanyaan-jawaban bermakna.

? Bolehkah satu orang mengerjakan proyek ini?

Tidak dianjurkan. Proyek dirancang untuk 2–3 orang. Jika ada alasan khusus, konsultasikan ke dosen sebelum hari ke-3.

? Bagaimana jika API LLM tidak bisa diakses?

Gunakan Ollama sebagai alternatif offline. Jika ada kendala teknis, hubungi dosen maksimal H+3 setelah proyek dimulai.

? Apakah UI-nya harus tampak profesional?

Tidak. Yang terpenting adalah fungsional. Streamlit sangat direkomendasikan karena mudah dan hasilnya cukup rapi.

Pertanyaan lain? Kirim ke email dosen atau tanyakan di grup kelas. Tidak ada pertanyaan yang salah — lebih baik tanya lebih awal daripada salah arah selama 2 minggu!